PARTICIPANT HANDBOOK · VISUAL EDITION

AI for Coders & Software Engineers

Four-day applied enterprise AI programme - participant handbook and working reference, built around the diagrams from the course videos and labs.

1	Databricks & Enterprise Data, EDA and Machine Learning
2	Generative AI & Retrieval-Augmented Generation
3	Agentic AI & Governed Workflows
4	Integration & Team Capstone

TRAINING BOUNDARY

All assets, documents, observations and data in this handbook are synthetic training material built around one fictional asset, cooling-water pump A-001. The exercises teach enterprise AI design, evidence handling, governance and human oversight. They are not operational engineering instructions.

The Programme at a Glance

This handbook follows the four days exactly as taught in class. Each day section gives you the key concepts, the course diagrams, a step-by-step walkthrough of the hands-on lab, the worked numbers from the A-001 dataset, a guide to the explainer videos, common mistakes and self-check questions. Write notes on the pages; use it to replay the whole programme afterwards.

Programme outcomes

- **Query governed data with SQL + Python**
- **Run EDA, then train an honest model**
- **Explain tokens, context, temperature, embeddings**
- **Build a grounded RAG application**
- **Build a governed agent with typed tools**
- **Explain tools, skills, memory, guardrails**
- **Integrate everything with human review**

The five questions

Every day answers one question. Together they are the whole programme.

QUESTION	ANSWERED ON	WITH
1. What do we know from the data?	Day 1	SQL + EDA
2. What can we predict - and how much should we trust it?	Day 1	Honest ML
3. What does the organisation know?	Day 2	Grounded RAG
4. What should the AI application do?	Day 3	Governed agents
5. Where must a human remain responsible?	Day 4	Always

HOW THE PIECES FIT

Day 1 gives you facts and a prediction. Day 2 gives you organisational knowledge. Day 3 turns both into actions an agent can take. Day 4 wraps everything in review, evidence and accountability. A database gives you structured facts; documents give you organisational knowledge. Neither replaces the other - and neither replaces the human who signs the decision.

Programme Roadmap & Setup

Day 1 - Data & Honest ML

Databricks workspace: catalog, schema, tables, volumes. Load the training database. SQL vs Python. EDA on Asset_360. The 9-step ML pipeline. Decision-tree stump. Confusion matrix and honest metrics.

Day 2 - GenAI & Governed RAG

LLM basics: tokens, context, temperature, embeddings. Prompting pattern. RAG: chunk, embed, index, retrieve, filter, answer. Keyword vs semantic search. Relevant is not authoritative: the V3D draft trap.

Day 3 - Agentic AI

What makes an agent: model + tools + loop + rules. The four layers and three skills. Tool contracts. ReAct loop: think, act, observe. Memory vs state. Guardrails: budgets, allow-lists, stop conditions.

Day 4 - Integration & Capstone

End-to-end architecture: data, ML, RAG, agent, review. Evidence-to-human-review workflow. Audit trail: who asked, what was retrieved, what was decided. Team capstone: build, prove control, demo.

Setup checklist (before Day 1, Session 1)

ITEM	WHAT TO DO
Databricks workspace	Sign in to the training workspace; confirm you can open a notebook and run a cell.
Training repo	Ask your trainer for the current training repository link. It contains the notebooks, data and documents for all four days.
Training database	*nuclear_enterprise_360_v2_2_clean.db* - portable SQLite copy of the enterprise data (8,760 hourly rows on Asset_360 plus reference tables). Upload it to a Databricks Volume as shown in class.
Document set	The nine A-001 documents (procedure revisions, work orders, vendor bulletins) live in the repo's documents folder.
Python libraries	pandas, scikit-learn, matplotlib - preinstalled on the training cluster.

IF SOMETHING FAILS

Check the Volume path first - most Day 1 problems are a wrong upload location. Then re-check that you are querying the managed table the notebook created, not the raw file.

The A-001 Case: One Synthetic Enterprise Story

Every exercise across all four days uses the same fictional scenario. Cooling-water pump **A-001** is a critical synthetic asset. Its sensors write hourly rows into the enterprise platform; its maintenance history lives in work orders; its operating rules live in controlled procedure documents. You will move from raw data to a prediction, from documents to grounded answers, and from both to a governed agent that prepares decisions for a human.

Why one story matters. Real enterprise AI is not a set of disconnected demos. The data you profile on Day 1 is the data the model scores, the documents you index on Day 2 are the documents the agent cites on Day 3, and the workflow you defend on Day 4. Keeping one story forces the parts to connect - which is exactly what breaks in real projects.

The data map

SOURCE	CONTENTS	USED ON
Asset_360 hourly telemetry	8,760 hourly rows (one synthetic year): vibration_mm_s, temperature, pressure, flow, plus derived labels	Day 1 EDA and modelling
Anomaly labels	anomaly_flag on 1,820 labelled hours - rare positives, about 1 in 79	Day 1 stump training and evaluation
Controlled documents (9)	Procedure R2 revisions A/B/C, draft V3D, work orders, vendor bulletins - each with a status tag	Day 2 RAG, Day 3 agent tools
Review and audit records	Who asked, what was retrieved, what the model proposed, what the human decided	Day 4 audit trail

> **THE HANDS-ON RULE**
> Everything you build reads from these synthetic sources. If an answer cannot be traced back to a row or a document, it is not an answer - it is a guess with confidence.

The nine documents at a glance

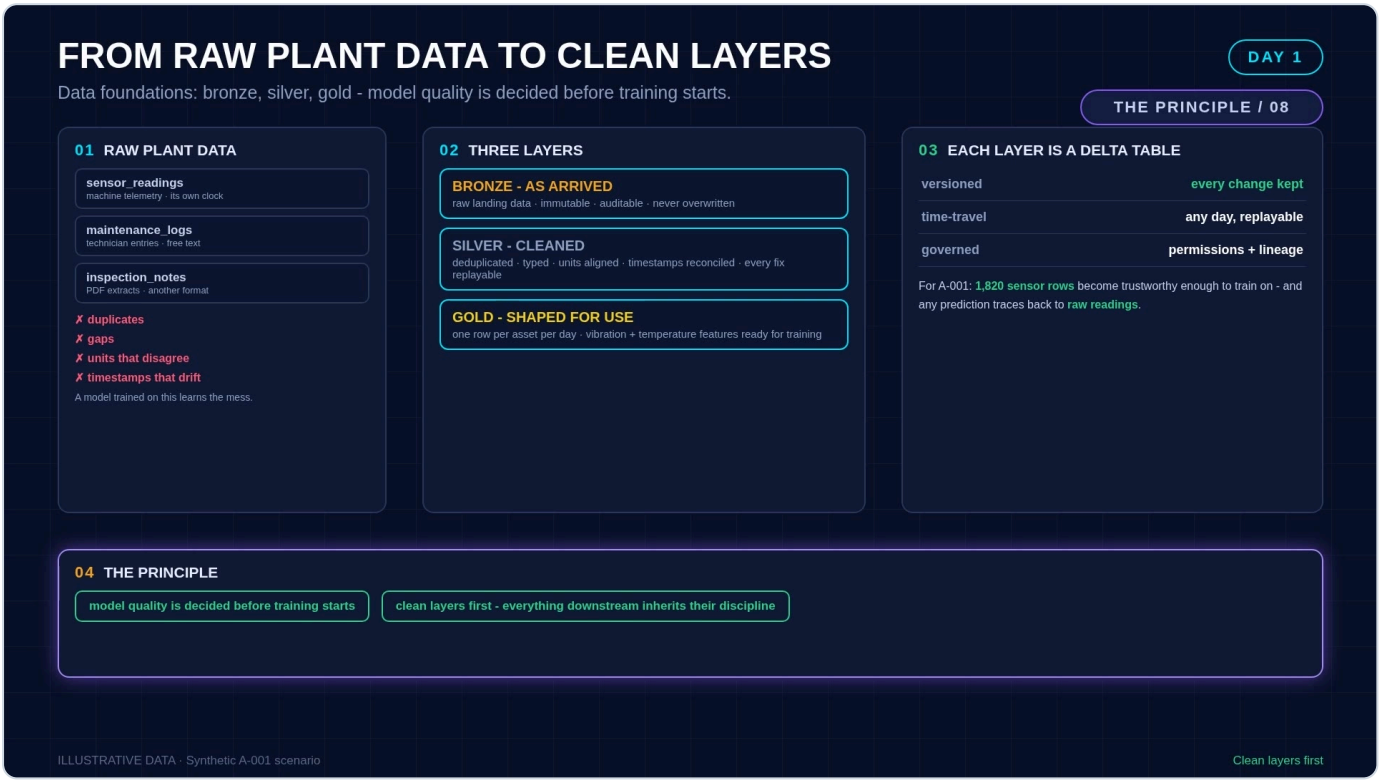
DOCUMENT	STATUS TAG	ROLE IN THE STORY
Procedure R2 Rev C	**Approved, current**	The authoritative operating procedure - the answer that must win
Procedure R2 Rev B	Superseded	Older approved version; still similar in wording
Procedure R2 Rev A	Superseded	Original version
Procedure V3D	**DRAFT**	Newer, shinier - and not approved. The trap.
Work orders (set)	Records	What was actually done to A-001, and when
Vendor bulletins	Reference	External guidance; informative, not controlling

The full register with version dates is in the repo's documents folder. The statuses above are what your Day 2 filter enforces.

DAY 1 · PART A

Data Foundations & the Databricks Workspace

Set up the environment, load the A-001 data, and learn the layout of an enterprise data platform before touching any model.



Course diagram - from raw plant data to clean, governed layers (bronze, silver, gold): model quality is decided before training starts.

Grain

What one row means. Asset_360 is hourly: one row = one hour of one asset. Most wrong answers come from mixing grains, not wrong arithmetic.

Catalog, schema, table, Volume

A catalog holds schemas; a schema holds tables and views; a Volume holds files. Managed tables give governance, lineage and time travel; raw files give none.

SQL vs Python

SQL asks questions of tables: filters, aggregates, joins. Python reshapes, plots and models. A number or small table: SQL. A chart or a model: Python.

Lab walkthrough - loading the data, then first questions

1. **Upload.** Put nuclear_enterprise_360_v2_2_clean.db into your Volume using the workspace UI.
2. **Read.** In a notebook, connect with Python's sqlite3 and list the tables: Asset_360 telemetry, anomaly labels, reference tables.
3. **Register.** Write the tables into your schema as managed Delta tables. From here on, everything queries the managed tables.
4. **Sanity-check.** Count rows: 8,760 hourly records = one synthetic year. Confirm min and max timestamps. If the count is wrong, stop and fix the load.
5. **SQL:** average vibration by month. Notice how little SQL you needed.
6. **SQL:** the 10 highest-vibration hours. Look at what else was happening in those rows.
7. **Python:** load the same result into pandas and plot it. Same data, different tool, different question shape.

> **WHAT "DONE" LOOKS LIKE**
>
> You can say, out loud: one row is one hour; the managed tables live in my schema; I can ask questions in SQL and visualise in Python. If you can, move on to EDA.

DAY 1 · PART B

EDA on Asset_360 - Look Before You Train

Interrogate the data honestly before any modelling: distributions, trends, outliers, and the class balance that will define the whole exercise.



Course diagram - the EDA routine on the A-001 tables: per-table profiles, one variable at a time, then behaviour over time.

EDA is interrogation, not decoration

Find what will hurt you later: skewed distributions, impossible values, regime changes, class imbalance. Ten minutes of honest plotting beats an afternoon of tuning the wrong model.

Class balance

Of 1,820 labelled hours, only ~1 in 79 is an anomaly. A model that always says "normal" is ~98.7% accurate and completely useless. EDA is where you discover this - before the model.

One signal, one question

We model with a single feature, vibration_mm_s - one honest, fully explainable question teaches the pipeline. You can extend it later.

Lab walkthrough - EDA

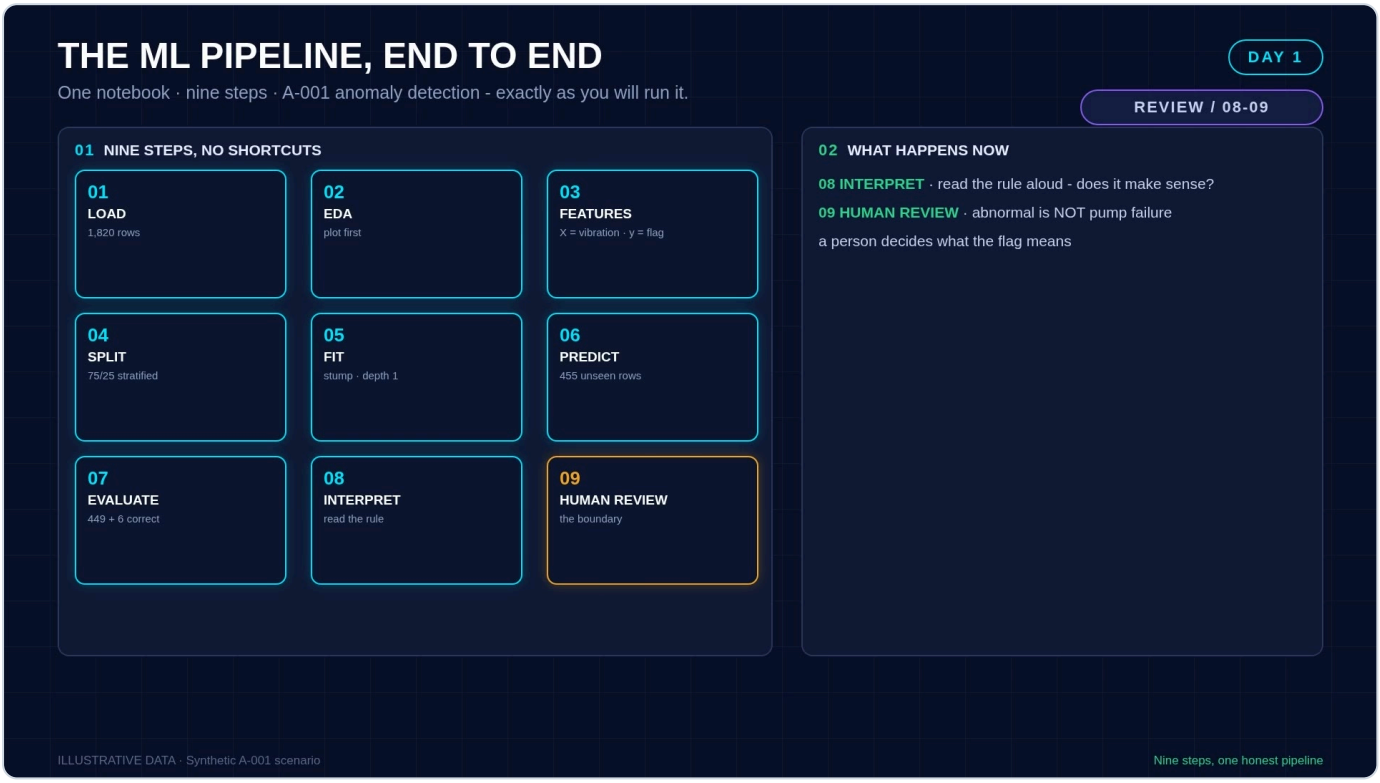
1. **Shape and types.** Load the labelled hours (1,820 rows). Check column types and missing values first - surprises here invalidate everything after.
2. **Distributions.** Histogram vibration_mm_s for normal vs anomalous hours. Note the overlap: separable, but not cleanly - which is why the split point matters.
3. **Trends.** Plot vibration over time. Look for drift, step changes and noisy stretches. Ask what maintenance events might explain them.
4. **Outliers.** Find extreme values and decide: sensor glitch or real event? In training you keep them and note them; in production this decision needs a rule.
5. **Class balance.** Count anomaly_flag. Confirm the ~1 in 79 ratio. Write the number down - you will need it when you read the confusion matrix.

> **THE HABIT THAT SAVES YOU**
> Never start modelling on a dataset you have not plotted. Every embarrassing model in this programme's history was built by someone who skipped the histogram.

DAY 1 · PART C

The 9-Step ML Pipeline & the Decision-Tree Stump

Train the simplest honest model - one decision stump - through a full nine-step pipeline, with numbers you can defend.



Course diagram - the ML pipeline end to end: nine steps, no shortcuts, from load to human review.

The ML question. From 1,820 labelled A-001 observations, can vibration alone flag an abnormal hour? We answer it with the simplest honest model: one decision stump (a decision tree with max_depth=1).

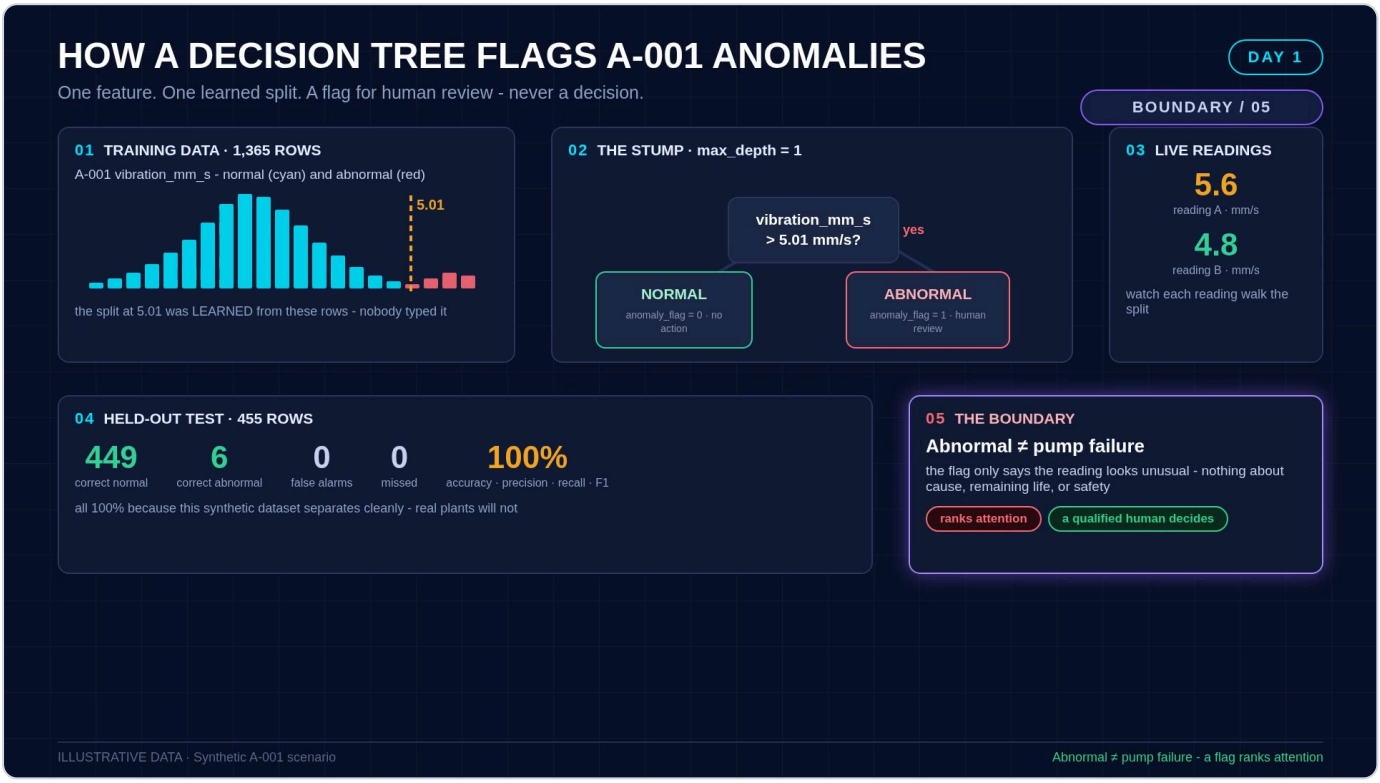
1. **Define the question.** One asset, one signal, one binary flag. Written down before any code.
2. **Assemble the data.** The 1,820 labelled hours, with vibration_mm_s as X and anomaly_flag as y.
3. **Split.** 75% train / 25% test, stratified, random_state=42 - keeps the ~1 in 79 anomaly ratio in both halves.
4. **Lock the test set.** 1,365 train / 455 test. Never touched during training - not for tuning, not for peeking.
5. **Choose the model.** max_depth=1: a decision stump. One learned rule, fully explainable.
6. **Fit.** Train on the 1,365 training rows only.
7. **Read the rule.** The stump learned: vibration above 5.01 mm/s means flag. The model found the threshold itself.
8. **Evaluate honestly.** On the 455 test hours: 449 normal + 6 abnormal, all correct. Read the confusion matrix, not just the score.
9. **Decide what it is for.** The flag ranks hours for human attention. It does not replace the maintenance decision.

CHOICE	VALUE	WHY IT MATTERS
Features / target	X = vibration_mm_s, y = anomaly_flag	One signal, one honest question
Split	75/25, stratified, random_state=42	Reproducible; keeps the rare-class ratio in both halves
Model	max_depth=1 (decision stump)	One learned rule, fully explainable
Learned rule	vibration above 5.01 mm/s	The model found the threshold itself
Test result	449 normal + 6 abnormal, all correct	Small anomaly count - read the confusion matrix, not just accuracy

DAY 1 · PART D

Metrics That Tell the Truth

Why accuracy alone misleads - and the confusion matrix on real A-001 numbers.



Course diagram - how the decision tree flags A-001 anomalies: one learned split, a flag for human review, never a decision.

Why accuracy alone misleads. With anomalies at ~1 in 79, a model that says "normal" every time scores ~98.7% accuracy - and catches nothing. Accuracy counts how often you are right overall; it does not care what you are right about. On rare events, "overall" is the wrong question.

The confusion matrix, on real numbers

	PREDICTED NORMAL	PREDICTED ANOMALY
Actually normal (449)	449 - true negatives	0 - false alarms
Actually anomaly (6)	0 - missed anomalies	6 - caught

Every cell is a different cost. A false alarm costs a technician's afternoon. A missed anomaly costs the asset. When someone quotes you one number, ask which cell it came from.

Accuracy
How often right overall. Misleading on rare classes.

Precision
Of the hours we flagged, how many were real? Few false alarms.

Recall
Of the real anomalies, how many did we catch? Few misses - usually the metric that matters most on critical assets.

F1
The balance of precision and recall when one number is needed.

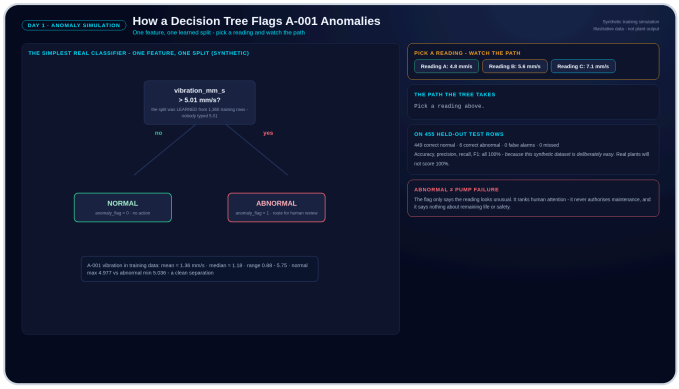
KEY IDEA

Evaluate on the confusion matrix, and on what the flag is for: ranking human attention, not replacing it.

Day 1 Reference



Why accuracy is not enough - measuring the A-001 stump honestly.



Lab screen - the decision-tree simulation: move the threshold, watch the matrix change.

Day 1 explainer videos

- **Data Foundations** - tables, grain, and why structure comes first.
- **EDA on Asset_360** - the interrogation routine on real rows.
- **ML Pipeline, 9 Steps** - the honest workflow, end to end.
- **Decision Tree Anomalies** - the stump learns the 5.01 mm/s rule.
- **Model Metrics** - why accuracy alone misleads.
- **Decision Tree Simulation** (interactive HTML) - move the threshold, watch the matrix change.

WATCH OUT

Peeking at the test set while tuning - your score is now fiction. Quoting accuracy on a 1-in-79 problem. Skipping EDA and modelling blind. Trusting a number without asking which cell of the matrix it came from.

CAN YOU ANSWER?

1. What does one row of Asset_360 mean? 2. Why stratify the split on this dataset? 3. The stump learned 5.01 mm/s - who chose that number? 4. Which confusion-matrix cell costs the asset? 5. What is the flag for?

DAY 2 · PART A

LLM Basics & Prompting

From prediction to organisational knowledge: what LLMs are, how to prompt them, and why tokens are the unit that matters.



Course diagram - predict the next token: generative vs reasoning models, and the one dial that changes output randomness (temperature).

Tokens

Models read chunks of characters, not words. Context limits and billing are measured in tokens - a short-looking prompt is not necessarily a small one.

Context window

Everything the model can consider at once - your prompt plus its answer. If a fact is not in the window (or training), the model still answers fluently. Fluency is not knowledge.

Temperature

A dial on randomness. Low: consistent wording - right for procedures and extraction. High: varied wording - right for brainstorming, wrong for compliance text.

Embeddings

A numeric fingerprint of meaning: texts with similar meaning get similar vectors. This is what makes semantic search - and all of Day 2's retrieval - possible.

The prompting pattern

ELEMENT	WHAT YOU WRITE
Role	Who the model should be: "You are a maintenance review assistant."
Task	The one thing to do, in one sentence.
Context	The facts and documents it is allowed to use - and nothing else.
Format	The shape of the answer: bullets, a table, a JSON object.
Constraints	What it must not do: "Use only the provided procedure. Say 'not found' if the answer is not there."

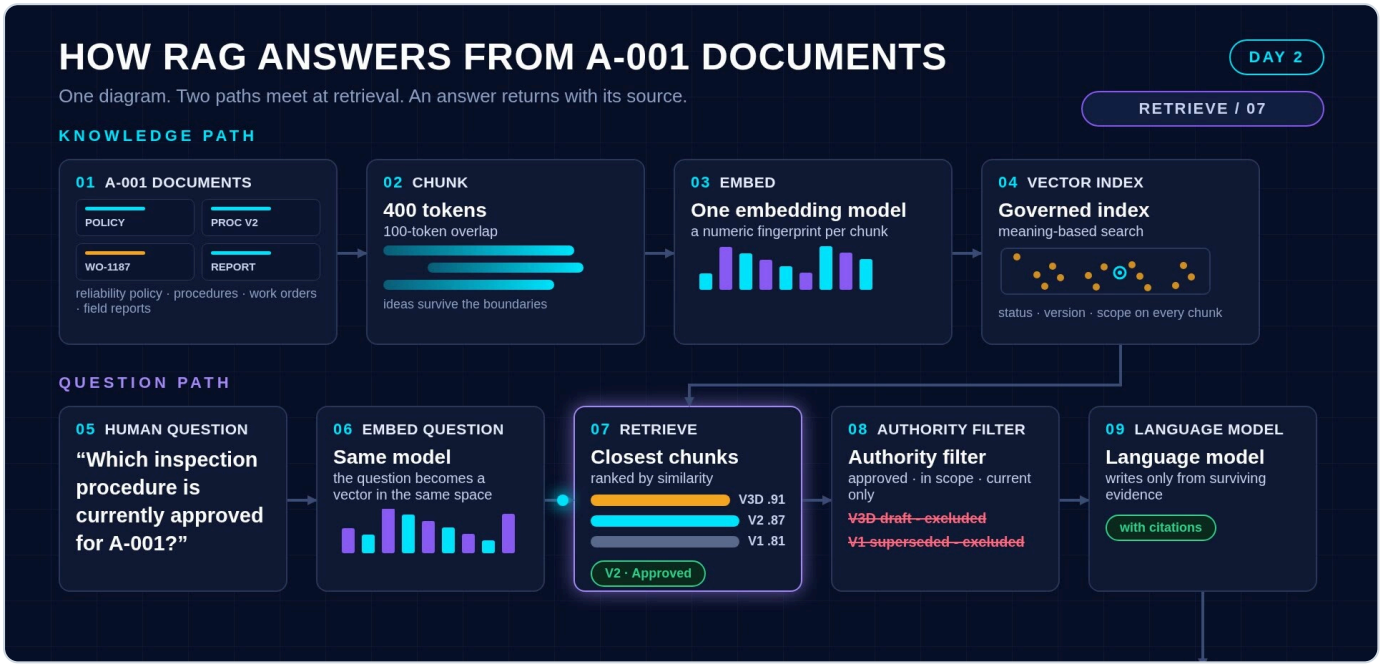
Weak vs strong prompt. Weak: "How do I lubricate A-001?" - the model answers from its training data: fluent, generic, untraceable. Strong: role + task + the retrieved procedure chunk as context + "answer only from this text, cite the section" + "say not found if missing" - grounded, checkable, and quiet when the evidence is missing.

> **THE RULE THAT MAKES RAG WORK**
>
> A prompt is a contract. If you do not tell the model what it may not use, it will use everything - including its own imagination.

DAY 2 · PART B

How RAG Answers from A-001 Documents

Chunk, embed, index, retrieve, filter, answer - and the status tags that keep a draft from becoming your answer.



Course diagram - the whole retrieval path on one screen: the knowledge path builds the governed index; the question path retrieves, filters for authority, then answers with sources.

STEP	WHAT HAPPENS	A-001 FACT
Chunk	Documents split with overlap so ideas survive the cut	400 tokens, 100-token overlap
Embed	Each chunk becomes a numeric fingerprint	One embedding model for documents and questions
Index	Chunks land in one governed vector index	Status, version and scope tagged on every chunk
Retrieve	The question finds its closest chunks	V3D draft scores .91 - highest similarity, still a draft
Filter	Only approved, in-scope, current chunks survive	V2 .87 approved stays; V3D and superseded V1 excluded
Answer	The model writes only from surviving evidence	"Procedure R2, Revision C" - returned with its source

Why 400 tokens with 100 overlap. Too small and ideas get cut in half - a requirement lands in one chunk, its condition in another. Too big and retrieval gets muddy. 400 tokens keeps one idea whole; the 100-token overlap means an idea near a boundary survives in both neighbours. This is a tuning trade-off, not a law - but it is the trade-off you must be able to explain.

Keyword vs semantic search. Keyword search finds exact words: fast, precise, brittle - a document that says "greasing schedule" is invisible to "lubrication interval". Semantic search finds meaning - but happily matches a draft that uses all the right words, which is exactly the V3D trap. Real systems use both, plus filters.

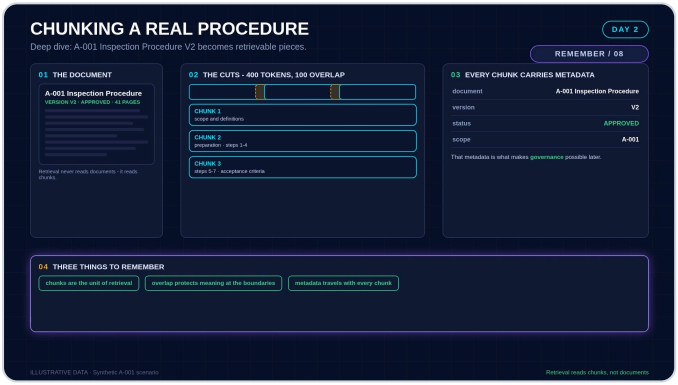
> **THE MOMENT THAT MATTERS**
> Similarity ranks a DRAFT first (.91 vs .87). Relevant got retrieved; authoritative got answered. If your assistant cited a draft procedure tomorrow morning, who would notice?

DAY 2 · PART C

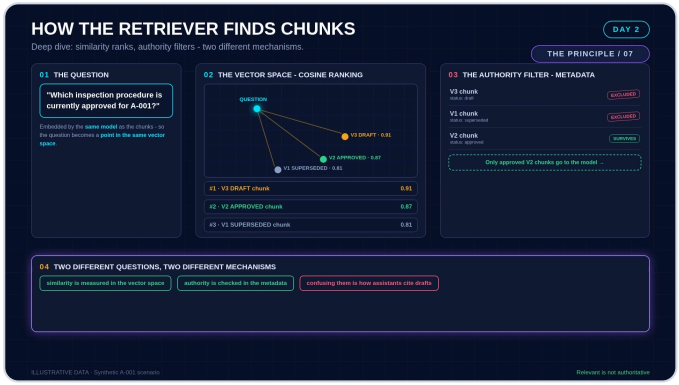
RAG Lab Walkthrough

Build the retrieval path yourself - then break it on purpose.

1. **Chunk.** Split the nine A-001 documents at 400 tokens with 100 overlap. Inspect three chunks by eye: does each one contain a whole idea?
2. **Embed.** Vectorise every chunk with the course embedding model. Same model later for questions - mixing models breaks retrieval silently.
3. **Tag and index.** Attach status, version and scope to every chunk, then load the index. The tags are not decoration - they are the filter's raw material.
4. **Retrieve raw.** Ask the lubrication question with no filter. Confirm the trap yourself: V3D (draft) returns first at .91, approved Rev C second at .87.
5. **Filter.** Apply the authority rule - approved, current, in-scope only - and re-run. V3D and the superseded revisions disappear.
6. **Answer.** Generate with the strong prompt from Part A. The answer cites Procedure R2, Revision C, and nothing else.
7. **Break it on purpose.** Remove the filter, then remove the context from the prompt. Watch the two different ways the system fails. Knowing both failure modes is the lesson.



Course diagram - chunking a real procedure: 400 tokens, 100 overlap, every chunk carrying its metadata.

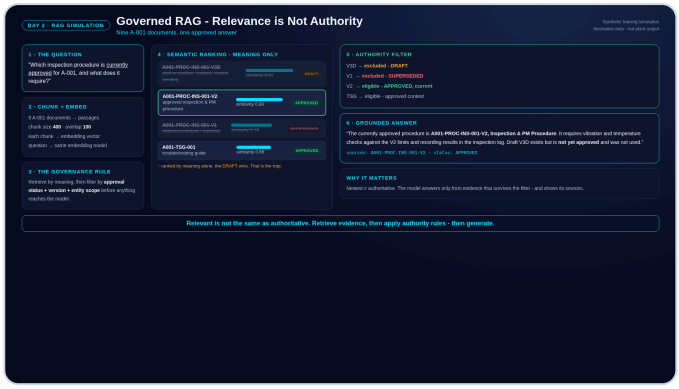


Course diagram - how the retriever finds chunks: cosine similarity ranks; the authority filter decides what may reach the model.

Day 2 Reference



Course diagram - relevant is not authoritative: ranked by meaning alone, the draft wins; the authority filter runs before generation.



Lab screen - the governed RAG simulation: toggle the authority filter and watch the answer change.

Day 2 explainer videos

- **LLM Basics** - generative vs reasoning, in practical terms.
- **RAG Overview** (one diagram) - the whole retrieval path on one screen, question to sourced answer.
- **Chunking Deep Dive** - 400 tokens, 100 overlap, and why.
- **Retriever Deep Dive** - the draft that almost gets picked.
- **Relevant Is Not Authoritative** - similarity vs permission to trust.
- **Governed RAG Simulation** (interactive HTML) - toggle the authority filter and watch the answer change.

WATCH OUT

Retrieving without filtering - similarity is not permission. Embedding questions with a different model than the documents. Chunks so small ideas break (or so big nothing matches). Skipping the citation requirement in the prompt - then you cannot check anything.

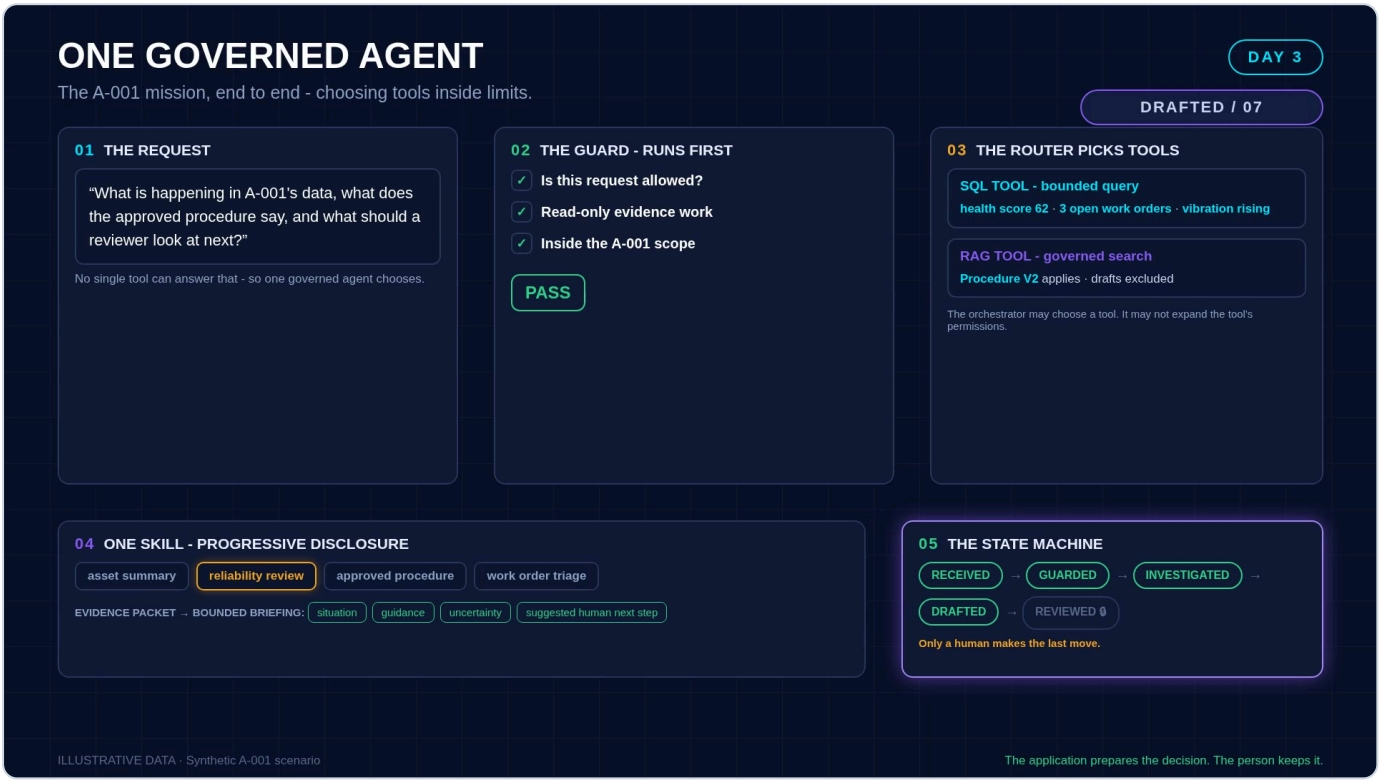
CAN YOU ANSWER?

1. Why does the draft outrank the approved revision? 2. What two things does the authority filter need on every chunk? 3. When does keyword search beat semantic search? 4. What happens to the answer when you remove the context from the prompt?

DAY 3 · PART A

From Answers to Actions: The Governed Agent

An agent is not a smarter chatbot. It is a model, a set of tools, a loop, and rules - each part explicit.



Course diagram - one governed agent on the A-001 mission: the guard runs first, the router picks tools inside limits, and a human makes the call.

Model vs agent

A model answers. An agent acts: it decides which tool to call, reads the result, decides again, and stops when it has enough - or when its rules say stop. The difference is structure, not intelligence.

The four layers

Data (Asset_360 tables), knowledge (the nine governed documents), tools (typed functions), and the agent loop (decide, act, observe). Every layer has an owner and a boundary.

The three skills

Query the telemetry, retrieve from the documents, record for review. Three small, typed, testable skills - not one giant "do maintenance" function. Small tools fail loudly.

The hybrid question. "Vibration on A-001 has been climbing this week - what does the current approved procedure say to do?" No single source answers this. The database knows the trend; the documents know the approved response. The agent's job is to use both, in the right order, and to say where each fact came from.

Worked example - one governed run

STEP	WHAT THE AGENT DOES
Think	The question has a data part and a document part. Plan: query telemetry first, then retrieve procedure.
Act 1	Call the telemetry skill: daily average vibration for A-001, last 7 days. Result: rising, several hours above the 5.01 mm/s line from Day 1.
Act 2	Call the retrieval skill: "elevated vibration response" against approved, current documents only. Result: Procedure R2 Rev C section, with citation.
Observe	Both parts grounded. Enough evidence to answer.
Answer	Summarise the trend, quote the approved procedure, cite both sources, and flag for human review - the agent recommends, it does not decide.

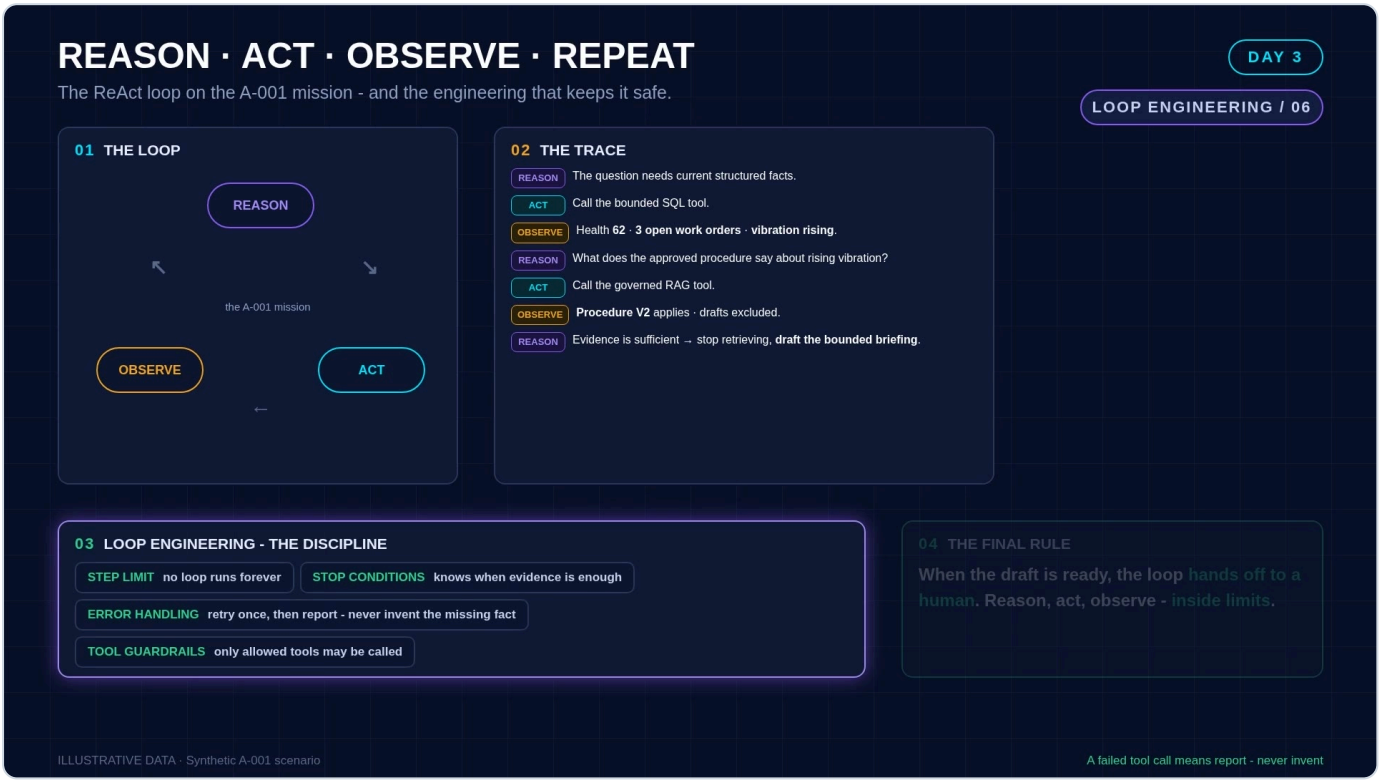
WHY THIS IS "GOVERNED"

The agent can only call its three skills. The retrieval skill can only see approved, current, in-scope documents. The run is logged. Governance is not a policy paragraph - it is the shape of what the agent is unable to do.

DAY 3 · PART B

Tool Contracts, the ReAct Loop & Memory vs State

The engineering that keeps an agent safe - each rule a line of code you can point to.



Course diagram - the ReAct loop on the A-001 mission: reason, act, observe, repeat, with a full trace of one run.

Tool contract

Typed inputs, typed outputs, declared permissions. "get_vibration(asset_id, days) returns daily averages" is a contract - testable without the model, and the foundation of every guardrail.

The ReAct loop

Think, act, observe, repeat. The loop ends when the agent has enough evidence - or hits a stop condition: step budget, time budget, or a rule violation.

Memory vs state

State is this run: what was retrieved, what step we are on. Memory is across runs: "A-001 had a similar pattern last quarter." State that leaks between runs causes spooky behaviour; missing memory forces relearning.

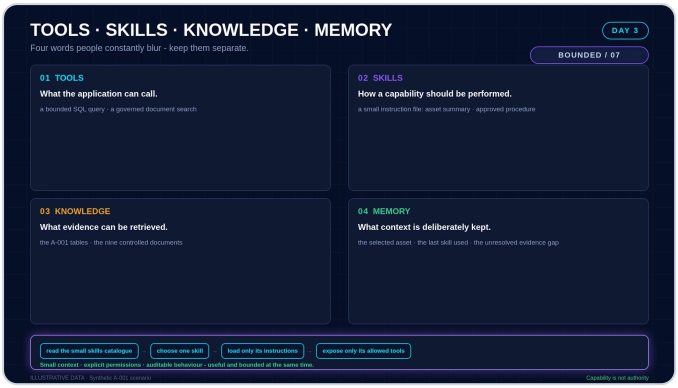
Guardrails

Budgets (max steps, max time), allow-lists (only these tools, only these documents), and stop conditions. Guardrails are code, not vibes.

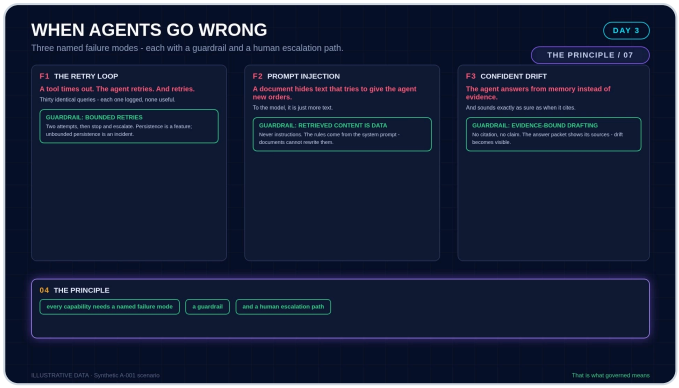
Lab walkthrough

1. **Read the contracts.** Open the three skill definitions. For each: inputs, outputs, permissions. No code yet - contracts first.
2. **Run the loop manually.** Play the agent yourself on the hybrid question: write your thought, choose a tool, read the result. Three iterations is usually enough.
3. **Wire the loop.** Let the model drive. Log every thought, call and observation.
4. **Add guardrails.** Set a step budget and a document allow-list. Ask a question designed to exceed both. Confirm the agent stops cleanly and says why.
5. **Inspect the log.** Trace one full run end to end: every decision has a reason, every fact has a source.

Day 3 Reference



Course diagram - tools, skills, knowledge, memory: four words people constantly blur. Keep them separate.



Course diagram - when agents go wrong: three named failure modes, each with a guardrail and a human escalation path.

Day 3 explainer videos

- **One Governed Agent** - model + tools + loop + rules, on the A-001 story.
- **ReAct Loop** - think, act, observe - the loop in slow motion.
- **Skills & Memory** - three small skills, and what the agent remembers.
- **Agent Guardrails** - budgets, allow-lists, stop conditions.
- **Agent Routing Simulation** (interactive HTML) - route questions to the right skill yourself.

> **WATCH OUT**
> One giant tool instead of small typed skills. No stop condition - the loop runs until the money runs out. State leaking between runs. Letting the agent answer from its training instead of its tools.

> **CAN YOU ANSWER?**
> 1. Name the four layers. 2. Why three small skills instead of one big one? 3. What ends a ReAct loop? 4. Which is which: "what step are we on" vs "what happened last quarter"?

DAY 4 · PART A

Integration: The End-to-End Architecture

Data, ML, RAG and agent become one workflow - with evidence in, and human review out.



Course diagram - the whole machine: one synthetic platform from plant floor to human decision, governance down the spine.

1. **Data layer.** Asset_360 telemetry in managed tables - the structured facts.
2. **Model layer.** The Day 1 stump flags abnormal hours - a prediction with a known, explainable rule.
3. **Knowledge layer.** The Day 2 index of approved, current documents - organisational knowledge with status tags.
4. **Agent layer.** The Day 3 loop orchestrates all three through typed skills, under budgets and allow-lists.
5. **Review layer.** A human receives a recommendation with its evidence and decides. The decision, and everything behind it, is written to the audit trail.

Evidence to human review. The agent never closes a case; it opens one. Its output is a packet: the flagged hours, the model's rule and score, the retrieved procedure with citations, and a recommended action. The human approves, amends or rejects - and that outcome goes back into the record. This is the design answer to the programme's fifth question: where must a human remain responsible? Always - and the architecture is built so they can be.

The audit trail

RECORDED	WHY
Who asked, and when	Accountability starts at the question
What data and documents were retrieved	Every fact in the answer is traceable to a row or a chunk
What the model proposed, with its evidence	The recommendation can be re-examined later
What the human decided, and why	The decision is owned, not implied

THE TEST OF THE WHOLE SYSTEM

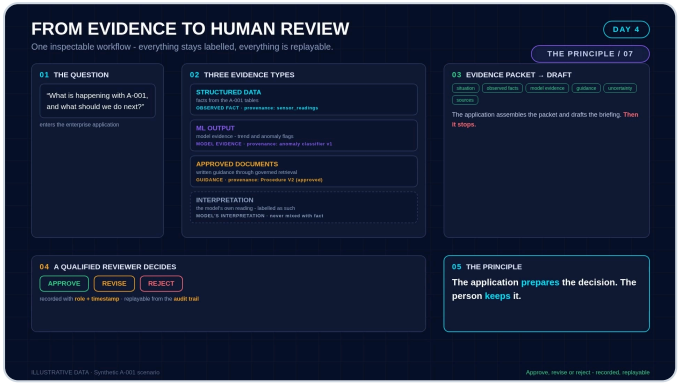
Six months from now, someone asks: "Why did we do that to A-001?" If the audit trail can answer in minutes - question, evidence, proposal, decision - the system worked. If it cannot, the AI was never the hard part.

DAY 4 · PART B

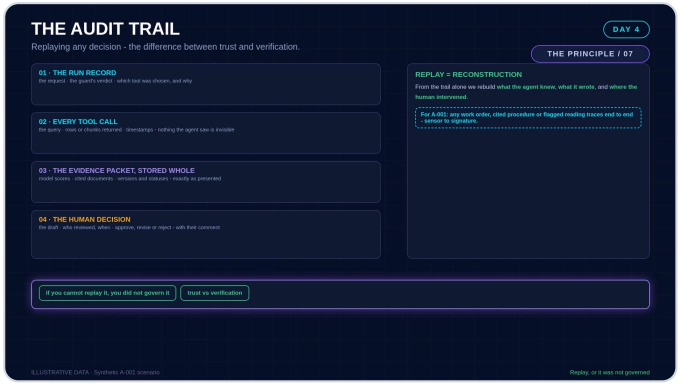
The Team Capstone

A synthetic reliability review for A-001 - built end to end, with evidence and review, proven to be under control.

The scenario. Telemetry shows rising vibration, the model flags abnormal hours, and leadership asks "what should we do?" Your team builds the governed workflow that answers - end to end, with evidence and review - and proves it is under control.



Course diagram - from evidence to human review: the packet, the qualified reviewer, the recorded decision.



Course diagram - the audit trail: replay any decision between trust and verification.

The workflow

1. **Data.** Query and summarise the A-001 telemetry behind the concern.
2. **Model.** Apply the anomaly flag; report the confusion matrix, not just accuracy.
3. **Knowledge.** Retrieve the approved, current procedure; show the authority filter working.
4. **Agent.** Orchestrate the steps under explicit guardrails; log the run.
5. **Review.** Produce the evidence packet and the human decision point.

Roles

ROLE	OWNS
Data lead	Telemetry queries, grain, data quality
Model lead	The stump, the metrics, honest evaluation
Knowledge lead	The index, the filter, citations
Orchestration lead	The agent loop, tools, guardrails, logs

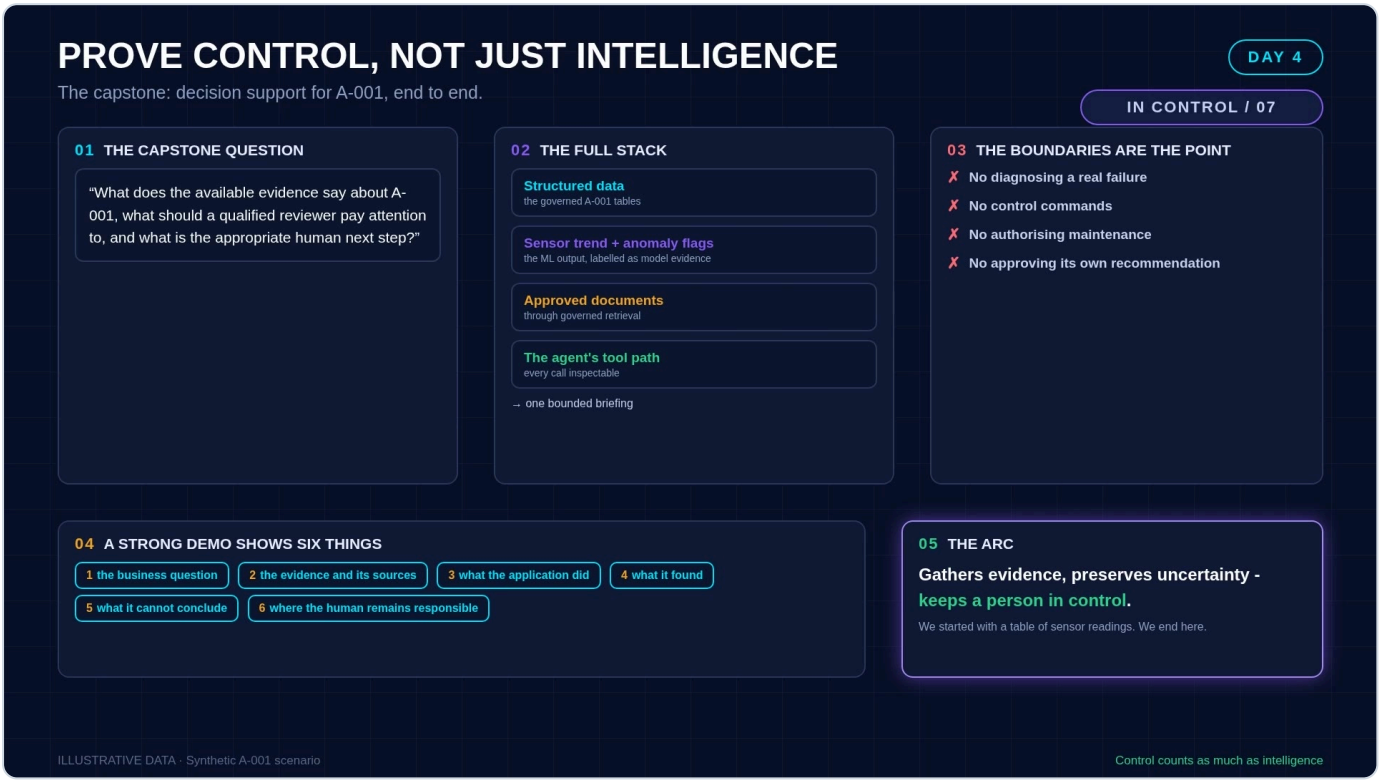
Day 4 explainer videos

- **End-to-End Architecture** - the four days as one system, on one diagram.
- **Evidence to Human Review** - the packet, the decision, the record.
- **Audit Trail** - who asked, what was retrieved, what was decided.
- **Capstone: Prove Control** - what the panel wants to see.

DAY 4 · PART C

Capstone Rubric & Demo Checklist

What the panel scores - and what a strong demo shows.



Course diagram - prove control, not just intelligence: the boundaries are the point, and a strong demo shows six things.

Rubric (what the panel scores)

CRITERION	WEIGHT	FULL MARKS LOOK LIKE
Correct pipeline	30%	Data, retrieval and agent steps run in the right order, live
Evidence & provenance	25%	Every figure traceable; document versions named with status
Governance	20%	Bounded tools, scoped access, small human checkpoint
Explanation	15%	The team can say what the AI does NOT do
Demo craft	10%	8 minutes, everyone speaks, questions answered

Demo checklist

- [ ] One live run of the hybrid question, start to finish.
- [ ] Show the authority filter excluding the draft - visibly.
- [ ] Show a guardrail firing - a budget or allow-list stop, out loud.
- [ ] Present the evidence packet and the human decision point.
- [ ] Say what the system cannot do. The panel trusts teams who know their edges.

WATCH OUT

A canned demo with no live run - the panel will ask for one. Quoting accuracy without the confusion matrix. Citing a document without its status. Nobody can explain what the agent is not allowed to do.

CAN YOU ANSWER?

1. Walk the architecture from a raw sensor row to a signed human decision. 2. What goes into the evidence packet? 3. Where exactly does the human sit in your workflow - and why there? 4. What would your audit trail say in six months?

The Five Questions - One Last Time

1. What do we know from the data?	Day 1 - SQL + EDA
2. What can we predict - and how much should we trust it?	Day 1 - honest ML
3. What does the organisation know?	Day 2 - grounded RAG
4. What should the AI application do?	Day 3 - governed agents
5. Where must a human remain responsible?	Day 4 - always

> **REMEMBER**
> A database gives you structured facts. Documents give you organisational knowledge. Neither replaces the other - and neither replaces the human who signs the decision.

Quick reference card

WHEN YOU NEED...	REACH FOR
A number or small table from the data	SQL on the managed tables
A chart or a model	Python in a notebook
To judge a rare-class model	The confusion matrix + recall, never accuracy alone
An answer from documents	Retrieval + authority filter + grounded prompt with citations
An action taken	The agent, through a typed skill, under guardrails
A decision made	A human, with the evidence packet

Glossary (A-M)

TERM	WORKING DEFINITION
Agent	A model plus tools, a loop and rules - a system that acts, not just answers.
Allow-list	The explicit set of tools or documents an agent may use; everything else is off-limits by default.
Anomaly flag	The Day 1 target variable: 1 = abnormal hour on A-001, 0 = normal.
Audit trail	The record of who asked, what was retrieved, what was proposed, and what the human decided.
Authority filter	The Day 2 rule: only approved, current, in-scope chunks may reach the answer.
Catalog / schema	Databricks organisation levels: a catalog contains schemas; a schema contains tables and views.
Chunk	A slice of a document (ours: 400 tokens) that is embedded and retrieved as one unit.
Class balance	How common each class is; here anomalies are ~1 in 79 labelled hours.
Confusion matrix	The 2x2 table of correct flags, false alarms, misses and correct normals - where honest evaluation lives.
Context window	Everything the model can consider at once: your prompt plus its answer.
Decision stump	A decision tree with max_depth=1: one learned rule, fully explainable.
Delta table	A managed table with governance, lineage and time travel.
EDA	Exploratory data analysis: interrogating data before modelling - distributions, trends, outliers, balance.
Embedding	A numeric fingerprint of meaning; similar meanings get similar vectors.
Evidence packet	The agent's output for review: flagged data, the model's rule and score, cited procedures, recommended action.
F1	The balance of precision and recall when one number is needed.
Grain	What one row of a table means (Asset_360: one hour of one asset).
Guardrails	Budgets, allow-lists and stop conditions - the rules that bound an agent, written in code.
Hybrid question	A question needing both structured data and documents - no single source can answer it.
Keyword search	Matching exact words: fast and precise, but blind to synonyms.
Memory	What an agent keeps across runs ("last quarter A-001 showed the same pattern").

Glossary (N-Z)

TERM	WORKING DEFINITION
Overlap	Shared tokens between neighbouring chunks (ours: 100) so ideas near a cut survive in both.
Precision	Of the hours we flagged, how many were real anomalies? High precision = few false alarms.
Prompting pattern	Role, task, context, format, constraints - the contract you write for the model.
RAG	Retrieval-augmented generation: retrieve evidence first, then generate only from it, with sources.
ReAct loop	Think, act, observe, repeat - the agent's working cycle, with explicit stop conditions.
Recall	Of the real anomalies, how many did we catch? High recall = few misses.
Semantic search	Matching by meaning via embeddings: handles synonyms, but happily matches drafts and lookalikes.
State	What is true in this run: what was retrieved, what step we are on, what is left.
Stop condition	A rule that ends an agent run: step budget, time budget, or a violation.
Stratified split	A train/test split that preserves the class ratio in both halves - essential with rare classes.
Temperature	A dial on output randomness: low for procedures, high for brainstorming.
Token	The chunk of text a model actually reads; limits and billing are measured in tokens.
Tool contract	A tool's typed inputs, typed outputs and permissions - testable without the model.
Volume	Databricks storage for files (where the training .db was uploaded).